AI Assisted Coding

Assignment8.3

Name: ch. koushik

Hallticketno:2303a51938

Batchno:19

Task 1:

Prompt:

GeneratePythonunittesttestcasesforvalidatingemailformatsbasedonthefollowing rules:

- Mustcontain@and.
- Mustnotstartorendwithspecialcharacters
- Mustnotcontainmultiple@symbols

Code&Output:

assignment_8.3.py

AI_Assistant_coding > assignment_8.3.py > ...

```python
# Task 1: Email Validation using TDD
import unittest
def is_valid_email(email):
    # Must contain exactly one '@'
    if email.
        retur
    # Must no                    haracters
    if email.startswith(('@', '.', '_')) or email.endswith(('@', '.', '_')):
        return False
    # Must contain '.' after '@'
    local_part, domain_part = email.split('@')
    if '.' not in domain_part:
        return False
    return True
class TestEmailValidation(unittest.TestCase):
    def test_valid_email(self):
        self.assertTrue(is_valid_email("test@example.com"))
    def test_missing_at_symbol(self):
        self.assertFalse(is_valid_email("testexample.com"))
    def test_missing_dot(self):
        self.assertFalse(is_valid_email("test@examplecom"))
    def test_multiple_at_symbols(self):
        self.assertFalse(is_valid_email("test@@example.com"))
    def test_starting_with_special_character(self):
        self.assertFalse(is_valid_email("@testexample.com"))
    def test_ending_with_special_character(self):
        self.assertFalse(is_valid_email("testexample.com."))
if __name__ == "__main__":
    unittest.main()
```

(function) startswith: Any

Loading...

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding> & C:/Users/hariv/AppData/Local/Microsoft/WindowsApps/
python3.12.exe "c:/Users/hariv/OneDrive/Documents/SRU/3 year II sem/AI_Assistant_coding/assignment_8.3.py"
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding>
```

Explanation:

In this task, the AI began by creating test cases that included both valid and invalid email formats. The implementation was then developed to meet all the defined test conditions. The function verifies the presence of “@” and “.”, ensures that there is only one “@” symbol, and checks that the email does not begin or end with special characters. By followingtheTest-DrivenDevelopment(TDD)approach,thecorrectnessofthe solutionwas confirmed through successful test results.

Task2:GradeAssignmentusingLoops

Prompt:

Generateunittesttestcasesforafunctionassign_grade(score)withboundaryandinvalidinputs.

Code&Output:

assignment_8.3.py

AI_Assistant_coding > assignment_8.3.py > ...

```python
# Task 2: Grade Assignment using Loops
# Generate unittest test cases for a function assign_grade(score)
# with boundary and invalid inputs
import unittest
def assign_grade(score):
    # Validate input
    if not isinstance(score, (int, float)) or score < 0 or score > 100:
        return "Invalid score"
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
class TestAssignGrade(unittest.TestCase):
    def test_grade_A(self):
        self.assertEqual(assign_grade(95), 'A')
    def test_grade_B(self):
        self.assertEqual(assign_grade(85), 'B')
    def test_grade_C(self):
        self.assertEqual(assign_grade(75), 'C')
    def test_grade_D(self):
        self.assertEqual(assign_grade(65), 'D')
    def test_grade_F(self):
        self.assertEqual(assign_grade(50), 'F')
    # Boundary tests
    def test_boundary_A(self):
        self.assertEqual(assign_grade(90), 'A')
        self.assertEqual(assign_grade(89.9), 'B')
    def test_boundary_B(self):
        self.assertEqual(assign_grade(80), 'B')
        self.assertEqual(assign_grade(79.9), 'C')
    def test_boundary_C(self):
        self.assertEqual(assign_grade(70), 'C')
        self.assertEqual(assign_grade(69.9), 'D')
```

```python
    def test_boundary_D(self):
        self.assertEqual(assign_grade(60), 'D')
        self.assertEqual(assign_grade(59.9), 'F')
    # Invalid input tests
    def test_invalid_negative_score(self):
        self.assertEqual(assign_grade(-10), "Invalid score")
    def test_invalid_over_100_score(self):
        self.assertEqual(assign_grade(110), "Invalid score")
    def test_invalid_non_numeric_score(self):
        self.assertEqual(assign_grade("eighty"), "Invalid score")
if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding> & C:/Users/hariv/AppData/Local/Microsoft/WindowsApps/
python3.12.exe "c:/Users/hariv/OneDrive/Documents/SRU/3 year II sem/AI_Assistant_coding/assignment_8.3.py"
----------------------------------------------------------------------
Ran 12 tests in 0.001s

OK
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding>
```

Explanation:

The AI-generated test cases covered normal score ranges, boundary conditions, and invalid inputs to ensure thorough evaluation. The implementation assigns grades accurately using conditional logic and also checks the input type and valid range to prevent incorrect or misleadingresults.Sincealllthetestcasespasssuccessfully,thesolution’scorrectnessiswell confirmed.

Task3:SentencePalindromeChecker

Prompt:

Generateunittesttestcasesforcheckingwhetherasentenceisapalindromewhileignoringcase, spaces, and punctuation.

Code&Output:

```python
# Task 3: Sentence Palindrome Checker
# Generate unittest test cases for checking whether a sentence is a palindrome
# while ignoring case, spaces, and punctuation.
import unittest
import string
def is_palindrome(sentence):
    # Remove non-alphanumeric characters and convert to lowercase
    cleaned_sentence = "".join(
        char.lower() for char in sentence if char.isalnum()
    )
    return cleaned_sentence == cleaned_sentence[::-1]
class TestPalindromeChecker(unittest.TestCase):
    def test_palindrome_sentence(self):
        self.assertTrue(is_palindrome("A man, a plan, a canal, Panama"))
    def test_non_palindrome_sentence(self):
        self.assertFalse(is_palindrome("This is not a palindrome"))
    def test_empty_string(self):
        self.assertTrue(is_palindrome(""))
    def test_single_character(self):
        self.assertTrue(is_palindrome("x"))
    def test_palindrome_with_numbers(self):
        self.assertTrue(is_palindrome("12321"))
    def test_non_palindrome_with_numbers(self):
        self.assertFalse(is_palindrome("12345"))
    def test_palindrome_with_mixed_characters(self):
        self.assertTrue(is_palindrome("No 'x' in Nixon"))
    def test_non_palindrome_with_mixed_characters(self):
        self.assertFalse(is_palindrome("Hello, World!"))
if __name__ == "__main__":
    unittest.main()
```

```
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding> & C:/Users/hariv/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/hariv/OneDrive/Documents/SRU/3 year II sem/AI_Assistant_coding/assignment_8.3.py"
........
----------------------------------------------------------------------
----------------------------------------------------------------------
----------------------------------------------------------------------
Ran 8 tests in 0.002s

OK
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding>
```

Explanation:

The AI-generated test cases cover both palindromic and non-palindromic sentences to ensurepropervalidation.Intheimplementation,spacesandpunctuationareremovedusing regular expressions, and the text is converted to lowercase for consistency. The processed string is then compared with its reversed version. This approach guarantees accurate palindrome detection, regardless of the original formatting.

Task4:ShoppingCartClass Prompt:

GenerateunittesttestcasesforaShoppingCartclasswithadd_item,remove_item,andtotal_cost methods.

Code&Output:

```python
# Generate unittest test cases for a ShoppingCart class
# with add_item, remove_item, and total_cost methods.
import unittest
class ShoppingCart:
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, price):
        if item_name in self.items:
            self.items[item_name] += price
        else:
            self.items[item_name] = price
    def remove_item(self, item_name):
        if item_name in self.items:
            del self.items[item_name]
    def total_cost(self):
        return sum(self.items.values())
class TestShoppingCart(unittest.TestCase):
    def setUp(self):
        self.cart = ShoppingCart()
    def test_add_item(self):
        self.cart.add_item("Apple", 1.00)
        self.assertEqual(self.cart.items, {"Apple": 1.00})
    def test_add_multiple_items(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.add_item("Banana", 0.50)
        self.assertEqual(self.cart.items, {"Apple": 1.00, "Banana": 0.50})
    def test_remove_item(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.remove_item("Apple")
        self.assertEqual(self.cart.items, {})
    def test_remove_nonexistent_item(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.remove_item("Banana")  # Should not crash
        self.assertEqual(self.cart.items, {"Apple": 1.00})
    def test_total_cost(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.add_item("Banana", 0.50)
        self.assertEqual(self.cart.total_cost(), 1.50)
    def test_total_cost_empty_cart(self):
        self.assertEqual(self.cart.total_cost(), 0)
if __name__ == "__main__":
    unittest.main()
```

```
TERMINAL
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding> & C:/Users/hariv/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/hariv/OneDrive/Documents/SRU/3 year II sem/AI_Assistant_coding/assignment_8.3.py"
......
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding>
```

Explanation:

The AI-generated test cases thoroughly check item addition, item removal, and the calculation of the total cost. The class stores items and their prices in a dictionary, ensuring organizeddatamanagement.Itsmethodsproperlyupdatethecart'scontents,andthetotal amount is calculated by summing all the stored values. Since the implementation successfully passes all the generated tests, its reliability is well established.

Task5:DateFormatConversion Prompt:

GenerateunittesttestcasesforconvertingdateformatfromYYYY-MM-DDtoDD-MM-YYYY.

Code&Output:

assignment_8.3.py

AI_Assistant_coding > assignment_8.3.py > ...

```python
# Task 5: Date Format Conversion
# Generate unittest test cases for converting date format from YYYY-MM-DD to DD-MM-YYYY.
import unittest
def convert_date_format(date_str):
    if not isinstance(date_str, str):
        return "Invalid date format"
    parts = date_str.split('-')
    if len(parts) != 3:
        return "Invalid date format"
    year, month, day = parts
    if not (year.isdigit() and month.isdigit() and day.isdigit()):
        return "Invalid date format"
    if len(year) != 4 or len(month) != 2 or len(day) != 2:
        return "Invalid date format"
    return f"{day}-{month}-{year}"
class TestDateFormatConversion(unittest.TestCase):
    def test_valid_date(self):
        self.assertEqual(convert_date_format("2024-06-15"), "15-06-2024")
    def test_invalid_date_format(self):
        self.assertEqual(convert_date_format("15-06-2024"), "Invalid date format")
    def test_non_string_input(self):
        self.assertEqual(convert_date_format(20240615), "Invalid date format")
    def test_empty_string(self):
        self.assertEqual(convert_date_format(""), "Invalid date format")
    def test_extra_characters(self):
        self.assertEqual(convert_date_format("2024-06-15-01"), "Invalid date format")
if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding> & C:/Users/hariv/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/hariv/OneDrive/Documents/SRU/3 year II sem/AI_Assistant_coding/assignment_8.3.py"
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
PS C:\Users\hariv\OneDrive\Documents\SRU\3 year II sem\AI_Assistant_coding>
```

Explanation:

The AI-generated test cases check both correctly formatted and invalid date inputs to ensure proper validation. The implementation separates the input string into parts and rearranges them into the desired format. It also includes error handling to return a clear messagewhentheinputformatisincorrect.Sincealllthetestcasespasssuccessfully,the solution works as expected.

FinalConclusion:

This lab highlights the effectiveness of applying Test-Driven Development (TDD) with the support of AI. By creating test cases before writing the actual implementation, developers canbetterensurecorrectness,reliability,andpropervalidation.WhileAIhelpsspeedupthe process of generating tests, human review is still crucial to guarantee comprehensive, meaningful, and well-designed test coverage.